

# Comparison between different STORAGES used in Computer

A computer uses **multiple layers of memory**, each designed for a specific purpose. As we move from top to bottom, **speed decreases**, **size increases**, and **cost decreases**. This hierarchy has existed for decades because it keeps systems fast and efficient.

## 1. Registers

### What they are:

These are the **smallest and fastest** memory units, located *inside the CPU*. They hold data that the CPU is working with **right now**.

### Why they exist:

The CPU cannot afford to wait for RAM—it needs data *immediately*. Registers provide that instant access.

### Example:

Suppose you write:

$a = b + c$

CPU steps:

- It loads  $b$  into Register R1
- It loads  $c$  into Register R2
- Performs addition
- Stores the result in Register R3

Everything happens inside registers before sending results back to RAM.

### Types of registers:

- **IR (Instruction Register)** – stores the current instruction
- **PC (Program Counter)** – stores address of next instruction
- **ACC (Accumulator)** – stores intermediate results

## 2. Cache Memory

### What it is:

A small, high-speed memory between **CPU and RAM**.

It stores *frequently used instructions/data* so CPU doesn't need to fetch from RAM again.

### Why it exists:

RAM is fast, but not fast enough for the CPU.

Cache acts as a “middle man” to speed things up.

#### Levels:

- **L1 Cache** – inside CPU, fastest
- **L2 Cache** – slightly slower, larger
- **L3 Cache** – shared among cores, slower but larger

#### Real-life example:

When you repeatedly open Chrome, the CPU uses cached instructions, making launch faster.

## 3. Main Memory (Primary Memory / RAM)

#### What it is:

RAM stores **active programs and data** while the computer is running.

#### Key points:

- Fast but slower than cache
- Limited in size
- CPU communicates **directly** with RAM
- **Volatile** → data is lost when power goes OFF

#### Example:

When you open VS Code, your operating system loads its files into RAM.

When RAM is full, system becomes slow because it depends on slower disk storage.

## 4. Secondary Memory (Storage)

#### What it is:

Long-term storage used to keep data permanently.

#### Devices:

- HDD
- SSD
- Pen drives
- CDs
- Memory cards

#### Key points:

- **Non-volatile** → data remains even after shutdown
- Much **larger capacity**
- Much **slower** than RAM

### Example:

Your projects, movies, photos, and software installers are stored in secondary memory.

## 5. Comparison

### 1. Cost

- **Registers** – most expensive (high-end semiconductor fabrication)
- **Cache** – slightly cheaper
- **RAM** – affordable
- **Secondary Storage** – cheapest

### 2. Access Speed (Fast → Slow)

1. Register
2. Cache
3. RAM
4. Secondary Storage (HDD/SSD)

Example:

A CPU register operates in picoseconds, cache in nanoseconds, RAM in tens of nanoseconds, and SSD in microseconds.

### 3. Storage Size (Small → Huge)

- Registers → a few bytes
- Cache → a few MB
- RAM → GBs
- Secondary Storage → 256GB to many TBs

### 4. Volatility

- **Primary memory (Registers, Cache, RAM)** → volatile
- **Secondary memory** → non-volatile

### Memory Hierarchy

| Level | Memory Type       | Speed     | Size     | Cost           | Volatile |
|-------|-------------------|-----------|----------|----------------|----------|
| 1     | Registers         | Fastest   | Smallest | Most expensive | Yes      |
| 2     | Cache             | Very fast | Small    | Expensive      | Yes      |
| 3     | RAM               | Fast      | Medium   | Moderate       | Yes      |
| 4     | Secondary Storage | Slow      | Huge     | Cheap          | No       |

## Real-World Analogy

Think of memory like working in a kitchen:

- **Registers → The knife in your hand**  
Instant access, you use it constantly.
- **Cache → Ingredients pre-cut and kept near the stove**  
You grab them quickly.
- **RAM → Ingredients stored on the kitchen counter**  
You need them often but must move yourself to get them.
- **Secondary Storage → Fridge / pantry**  
Huge space but slow to access.